



University of Piraeus
School of Information and Communication Technologies
Department of Digital Systems

Level: Master's Programme of Studies

Network Security

Burp Suite

Supervising Professor: Prof. Christos Xenakis

Ioannis Kalaitzidis

ioannis_kalaitzidis@ssl-unipi.gr

Piraeus
26/12/2025

Abstract

This study examines the offensive approach (Red Teaming) in web application security. Using the PortSwigger-Web Security Academy platform and Burp Suite tool, vulnerabilities were analyzed and exploited in a controlled environment. Specifically, SQL Injection attacks were successfully implemented to retrieve hidden data, along with cross-site scripting attacks for executing arbitrary JavaScript code, demonstrating the critical importance of input validation mechanisms.

Contents

Introduction.....	1
Practical part of the assignment.....	2
Bibliography.....	16

Introduction

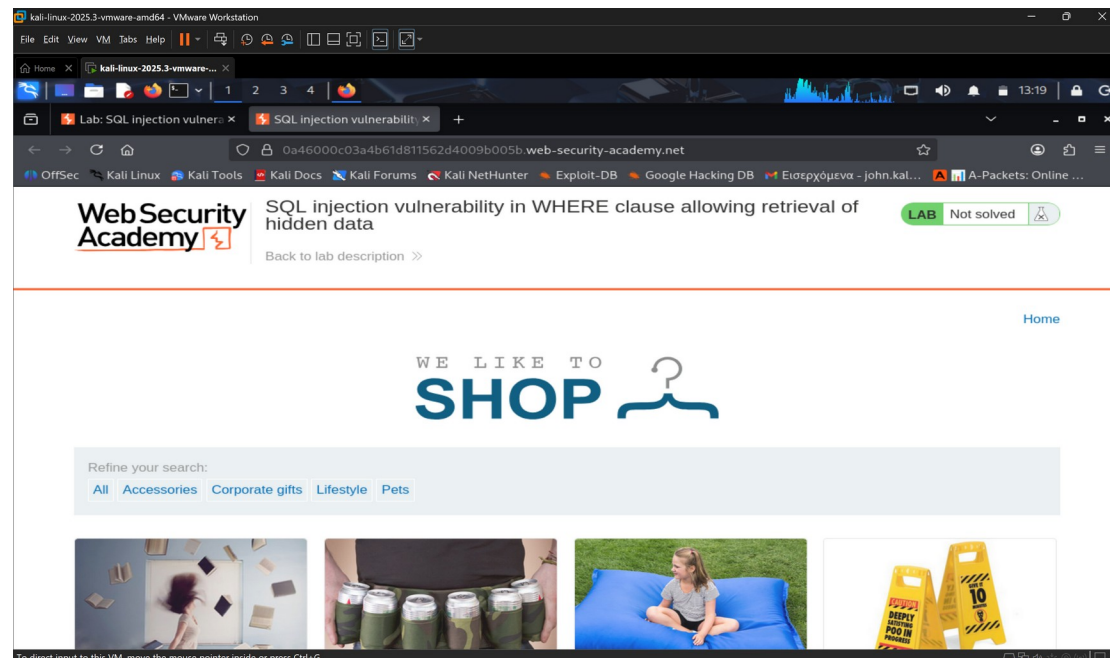
This voluntary assignment differs significantly from the previous one (Case 7), as it marks the transition from the defensive (Blue Team) to the offensive (Red Team) approach to information-systems security. Whereas in the previous scenario the primary goal was to harden the server through Firewall and WAF mechanisms in order to repel and block malicious traffic, in the current study we are called to adopt the mindset and techniques of the attacker. Specifically, the goal shifts from protection to actively breaching the application, seeking to identify and exploit vulnerabilities. For the practical implementation of the scenarios and the documentation of the SQL Injection and cross-site scripting attacks via the Burp Suite tool, the specialised educational environment of the PortSwigger Web Security Academy will be used. This choice was deemed optimal, as it provides direct access to realistic scenarios of vulnerable applications, allowing us to focus on analysing the attacks without the complexity and constraints that installing and configuring a local vulnerable server would impose. Finally, it is noted that the selected labs belong to the 'Apprentice' category, fully meeting the brief's requirement to document Low-difficulty vulnerabilities.

Practical part of the assignment

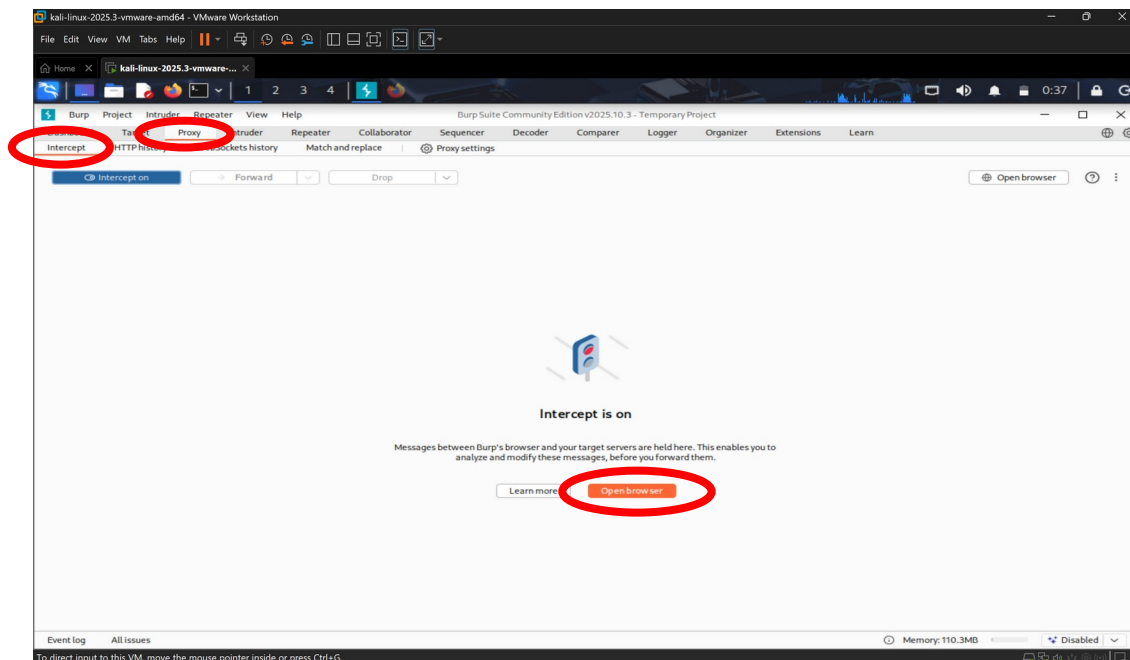
SQL injection

For the implementation of the assignment, the PortSwigger Web Security Academy educational platform was chosen, which provides a safe and controlled environment for carrying out web attacks. Specifically, we navigated to the SQL Injection topic and selected the lab titled:

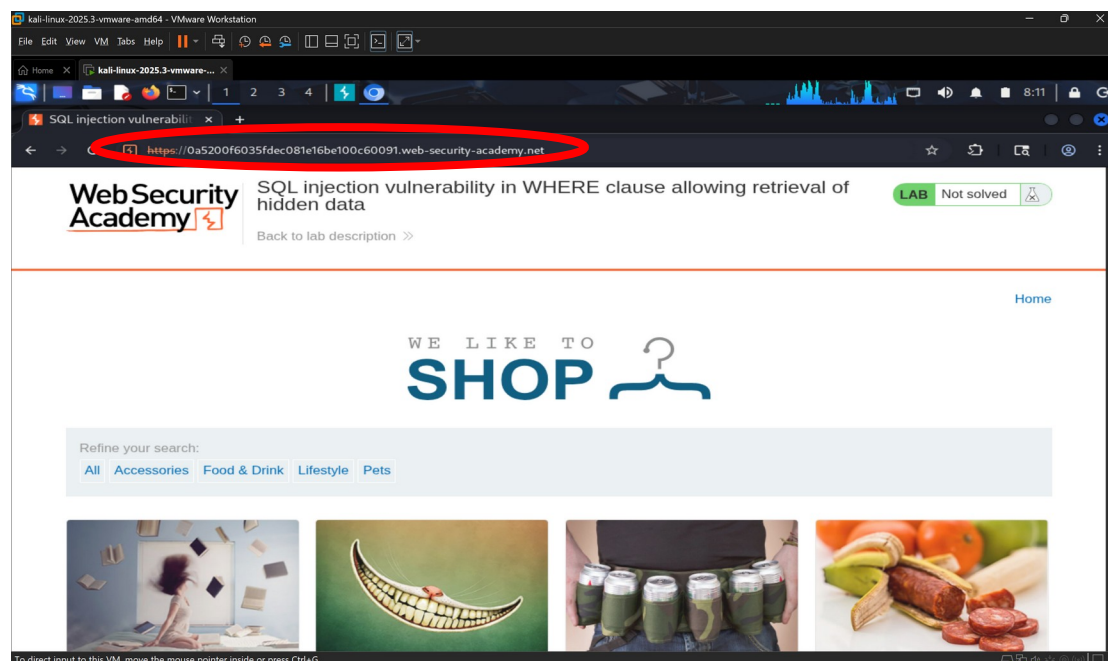
'SQL injection vulnerability in WHERE clause allowing retrieval of hidden data' (<https://portswigger.net/web-security/sql-injection/lab-retrieve-hidden-data>). The choice of this particular lab was deemed optimal for the first part of the exercise, as it simulates exactly the scenario we examined defensively in the previous assignment (Case 7). Whereas in the previous scenario our goal was to detect and block the 'OR 1=1' command via a WAF, in this scenario we act as attackers (Red Team). Our goal is the successful execution of the same logic (1=1), in order to manipulate the query to the database. In this way, we seek to bypass the application's checks and force the display of hidden data that would not normally be accessible to the end user.



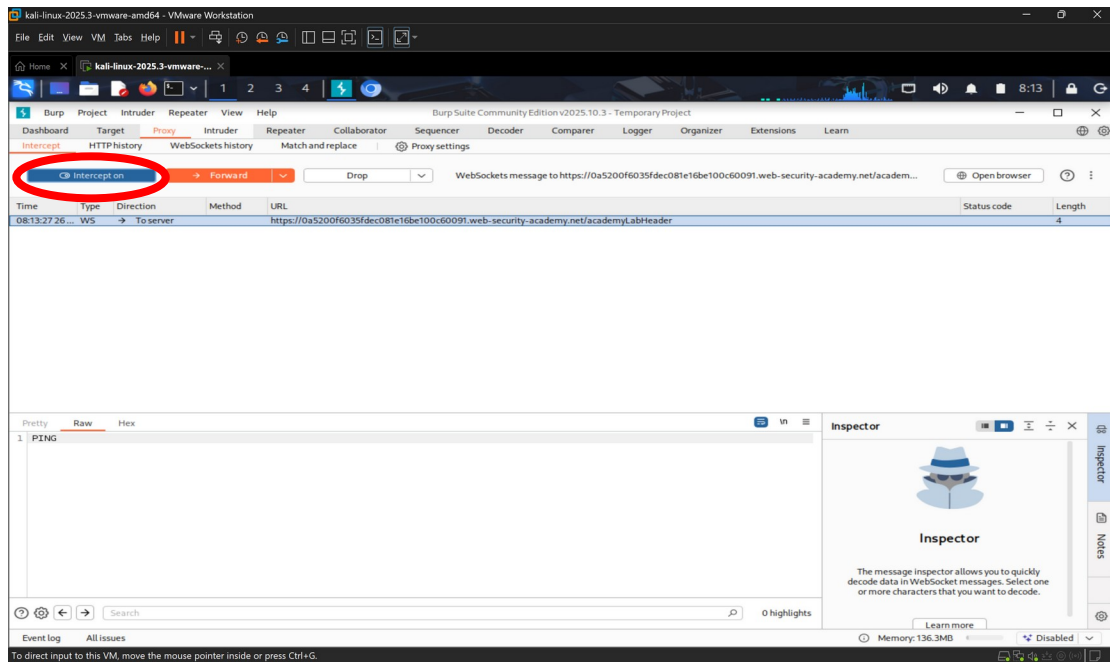
So we open Kali Linux and look for the Burp Suite application to launch it.



Once the application opens, we select the Proxy tab and then the Intercept tab. We click the Open Browser button, which opens the application's dedicated browser, and paste the address of this particular lab. We used Burp's built-in browser because it is ready to use. This way, we avoided complex network settings, since traffic automatically passes through the tool.



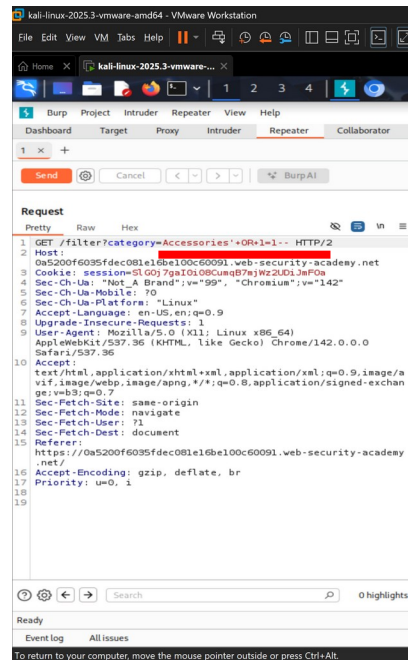
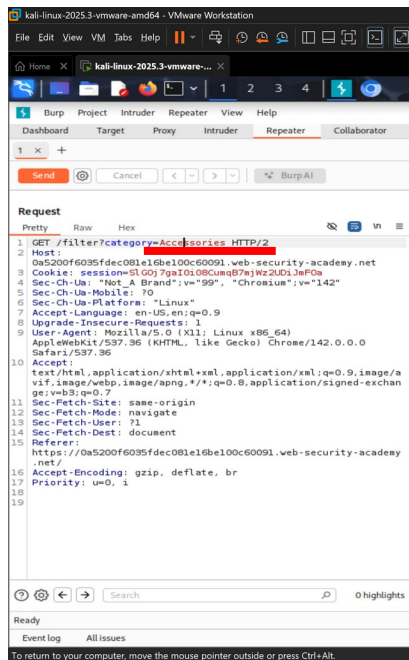
Now we return to the Burp Suite application and enable the Intercept on function as shown below.



Once we do this, we return to Burp's browser:

During the reconnaissance stage, we navigate to the lab's home page and interact with the application by selecting a product category (specifically the 'Accessories' category). This action triggers the sending of an HTTP request to the server. Our goal is to locate and intercept this particular request via Burp Suite, in order to check whether this parameter is vulnerable to SQL Injection.

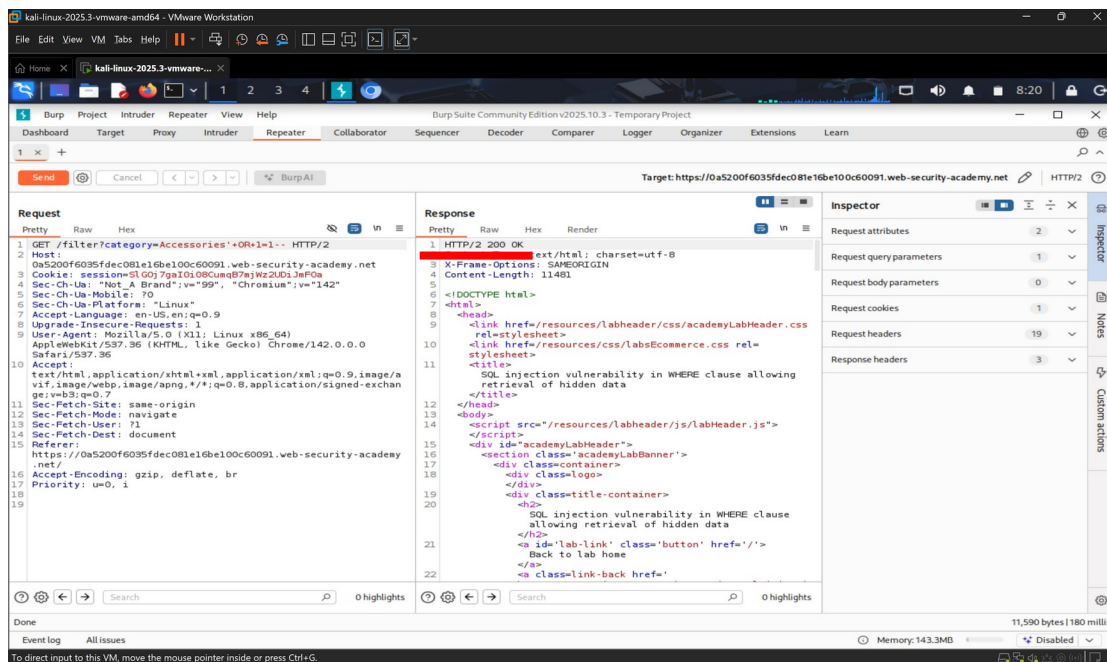
Once we enable the Intercept function, the HTTP request is successfully intercepted before it reaches the server. We confirm that it is the request calling the page with the results of the 'Accessories' category. Then, we right-click on the request and select 'Send to Repeater'. This transfers the request to a controlled testing environment, where we can modify the parameters and resend the request many times to check the system's reaction to the SQL Injection attack.



We locate the category parameter in the request line. We modify its value, replacing Accessories with the attack payload: "' OR 1=1 --".

Payload analysis:

- The single quote (') is used to close the string that the database expects.
- The condition OR 1=1 is a logical statement that is always true.
- The dashes (--) denote a comment in SQL, cancelling any part of the query that follows, so as to avoid syntax errors.

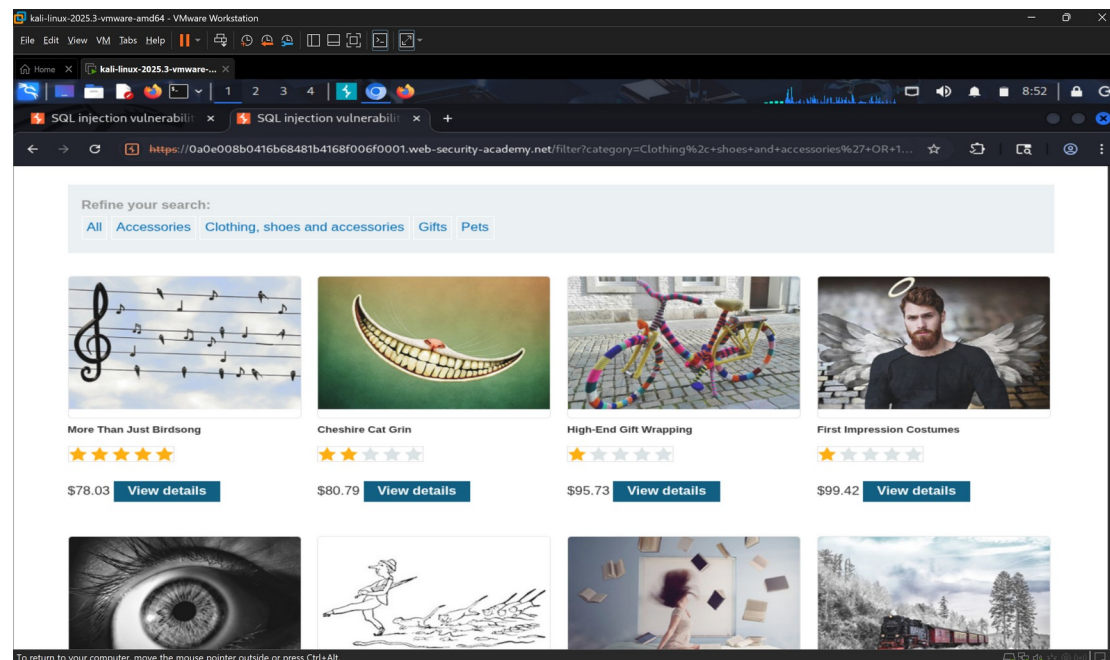


As shown in the underlined part of the image (Response), the server responded with a 200 OK success code. This indicates that the SQL Injection attack was executed correctly and did not cause a syntax error in the database. We also observe that the size of the response increased, as the web page now returns data for all the products in the database and not just for the specific category.

Having confirmed the success of the attack through the response codes in the repeater, the next step was visual verification in the browser. To achieve this, we disabled the Intercept function in Burp Suite's Proxy tab (which we had enabled earlier), so as to allow the free flow of data. We then used the modified URL (Copy URL) directly in the browser's address bar.

For the final verification, we compared the application's behaviour before and after the attack. In the initial search (image that follows), the application correctly filters the results, showing only the products belonging to the 'Clothing, shoes and accessories' category.

After injecting the OR 1=1 payload, we observe that the results now show all the products in the database, regardless of category, as well as products that were possibly hidden/unpublished.

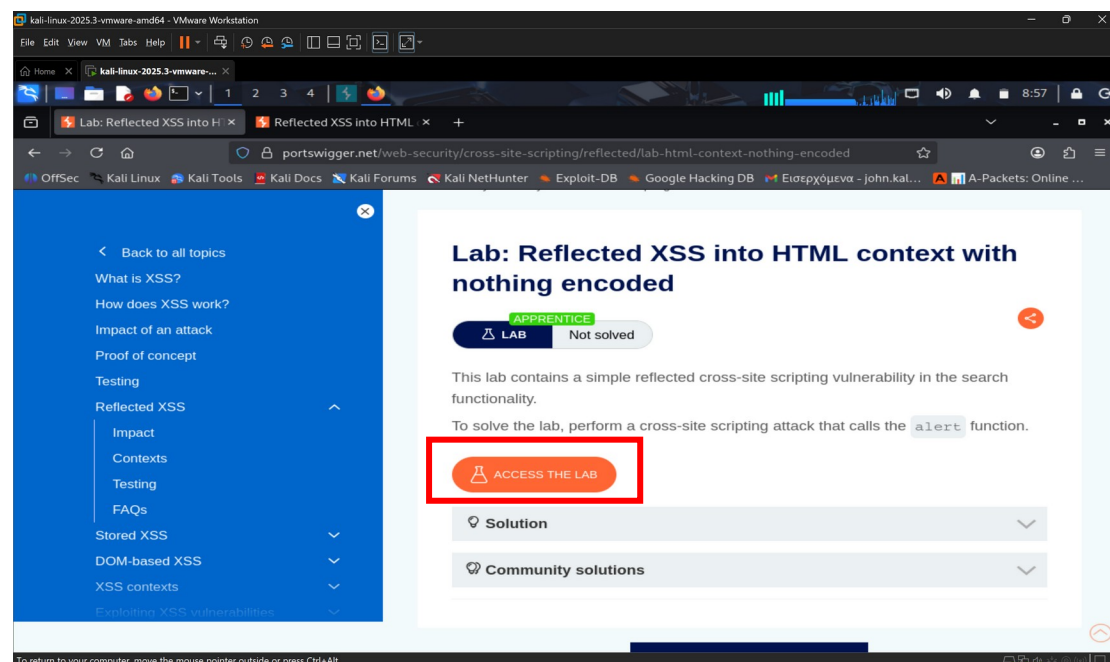


SQL Injection Section Conclusion

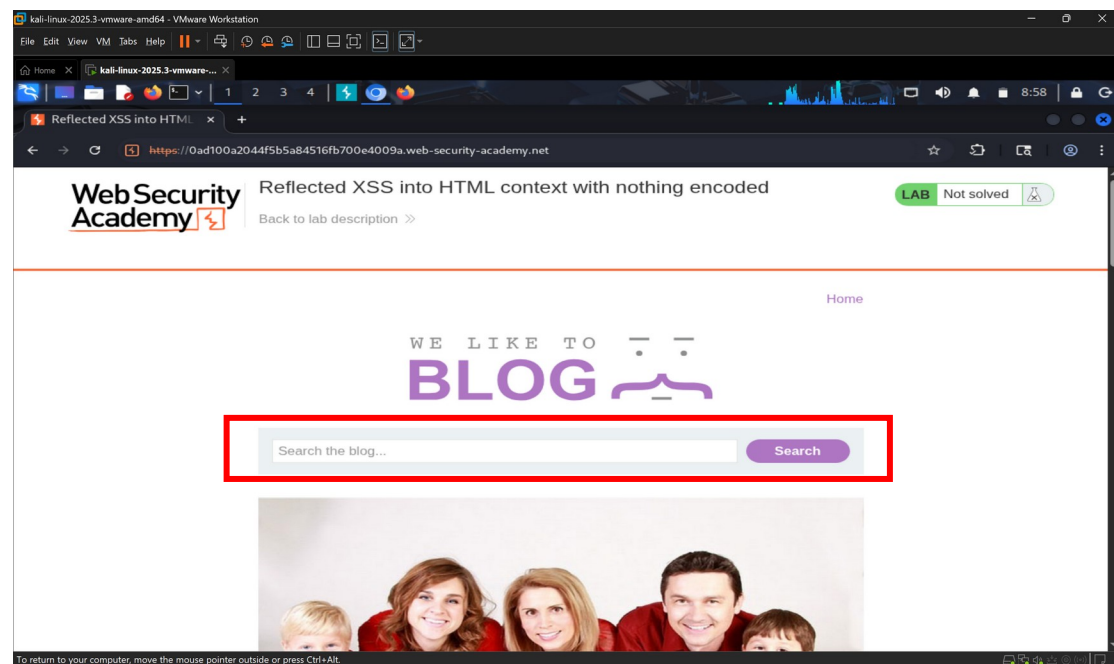
Through this attack, it was demonstrated that the application does not perform sufficient validation of the data entered by the user. By injecting the condition ' OR 1=1--, we managed to modify the logic of the SQL query to the database. Because the condition 1=1 always evaluates as true, the database stopped checking the product category and returned all the records of the table. This constitutes a serious breach of confidentiality, as it allows unauthorised users to gain access to data that the administrator had set as hidden or restricted.

Cross-site scripting attacks

For the second part of the assignment, we moved to the Cross-Site Scripting (XSS) attack category. Specifically, we selected the lab titled 'Reflected XSS into HTML context with nothing encoded' (<https://portswigger.net/web-security/cross-site-scripting/reflected/lab-html-context-nothing-encoded>). The goal of this exercise is to locate an input vector where the application 'reflects' the user's data back to the browser without proper filtering. This will allow us to inject and execute arbitrary JavaScript code.<https://portswigger.net/web-security/cross-site-scripting/reflected/lab-html-context-nothing-encoded>

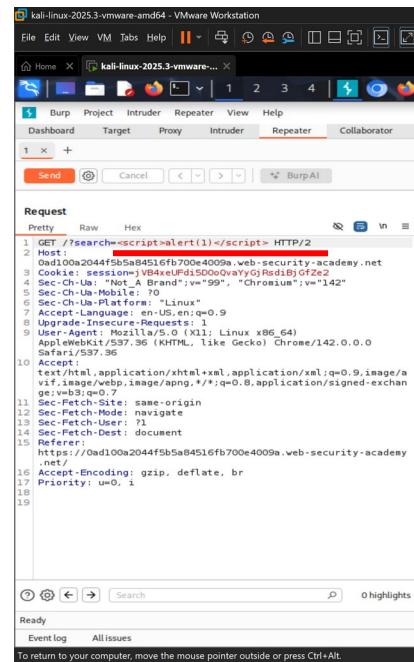
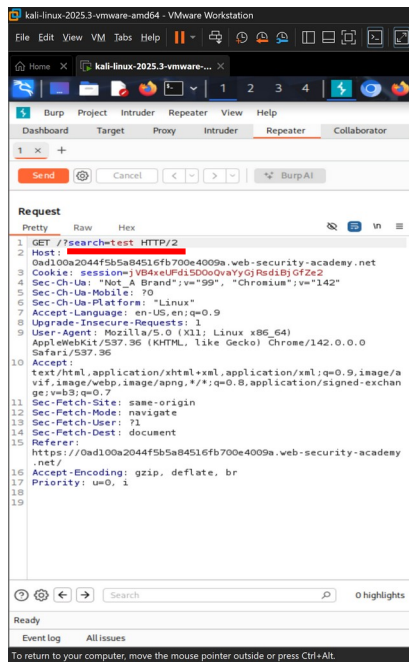


By clicking 'Access the Lab', we were taken to the environment of the vulnerable application, which simulates a blog. During the reconnaissance phase, we located the search bar.



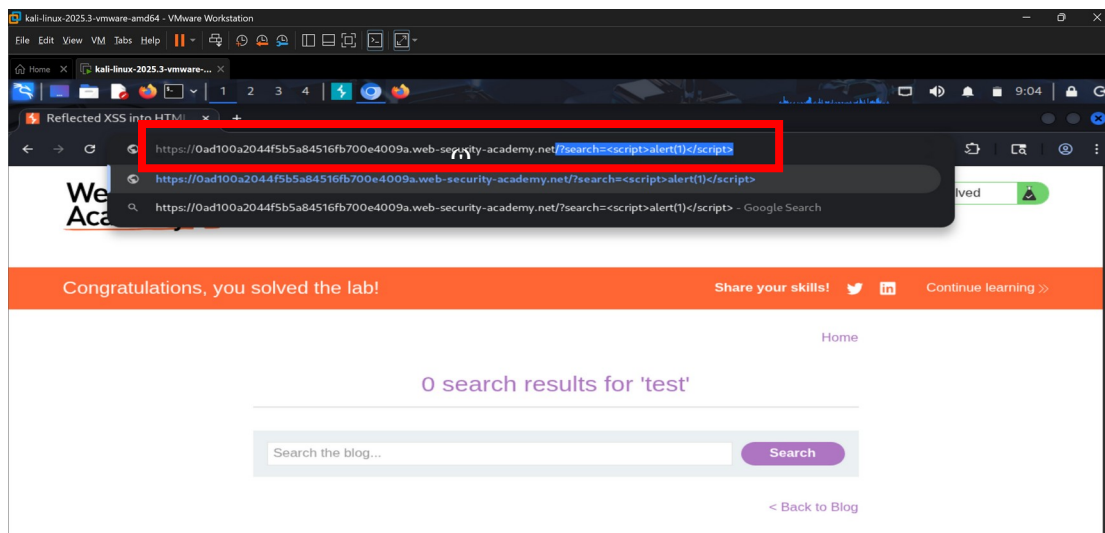
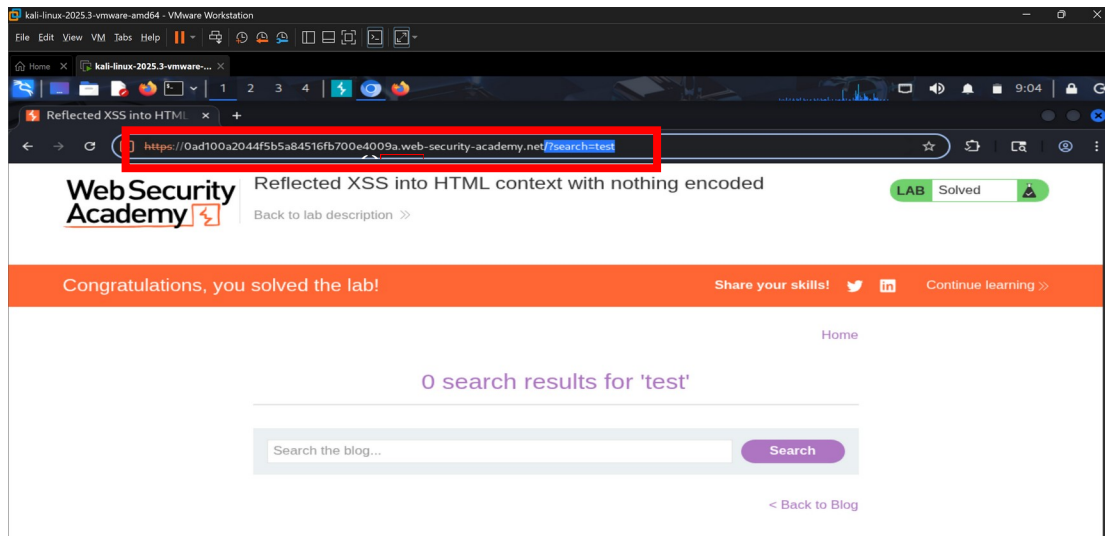
The search bar is the primary point of interest for a reflected XSS attack, as it is common for the data typed by the user to appear verbatim on the results page. The next step is to test the field via Burp Suite.

For traffic analysis, we followed the same methodology as in the previous section (SQL Injection). Specifically, we enabled the 'Intercept is On' function in Burp Suite and performed a search in the application to generate traffic. The request was successfully intercepted, allowing us to examine and modify it before it reached the server.

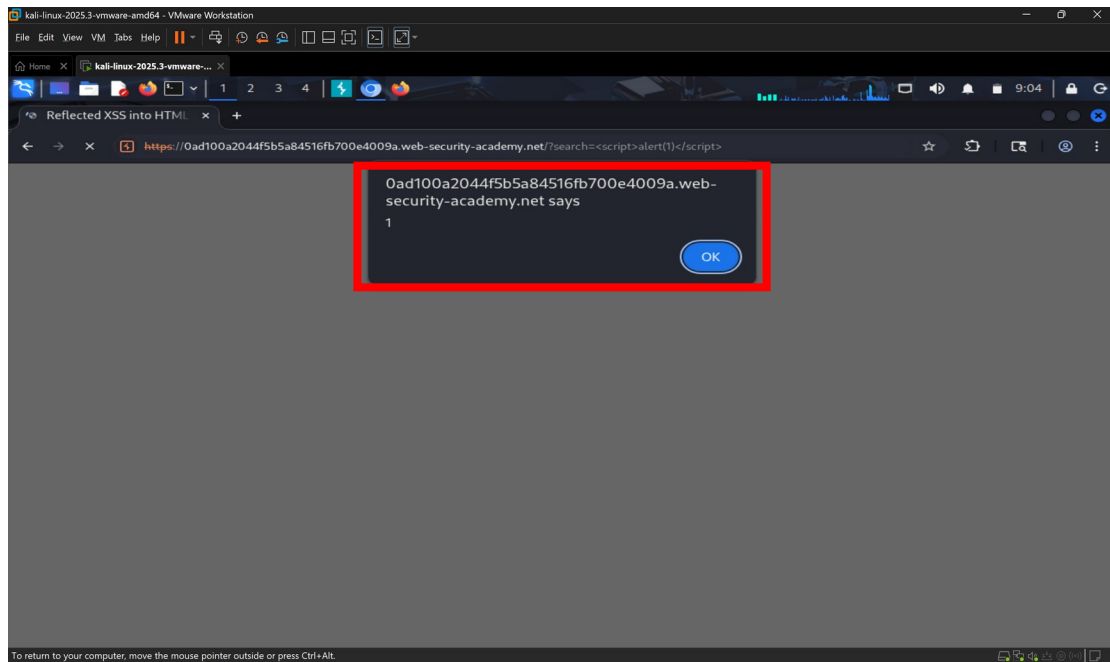


After locating the request carrying the search term, we sent it to the Repeater tool to perform tests without affecting the browser. In the Repeater, we replaced the original search term (test) with the classic XSS test payload '`<script>alert(1)</script>`'. The purpose of this script is to display a pop-up window (alert box) with the number '1', provided the browser executes the code.

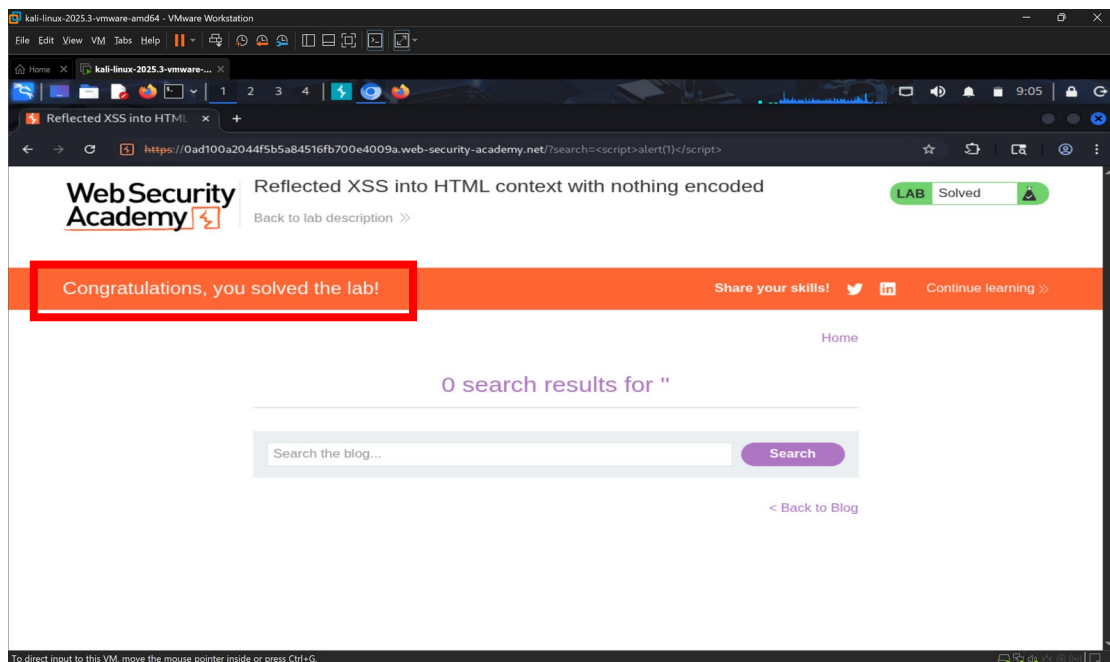
By clicking send, we received the server's response. Examining the HTML code on the right side (Response), we observed that the application returned our script exactly



Having confirmed the vulnerability in the repeater, we proceeded to the final execution of the attack in a real browser environment. First, we went to the Proxy tab and disabled the Intercept function (Intercept is Off), so as to allow the free flow of data. Then, we opened the browser and, in the address bar, manually modified the search parameter, injecting the payload `'/?search=<script>alert(1)</script>'`



By pressing enter in the address bar, the browser sent the request and received the response containing our script. As shown in the image, the browser interpreted the `<script>` tags as a command to execute and not as plain text. The result was the display of a pop-up alert with the value '1'. The appearance of this window is indisputable proof that the application is vulnerable to Cross-Site Scripting, as it allows the execution of JavaScript on the user's computer.



After closing the dialog box, the page loaded fully and the platform displayed the message 'Congratulations, you solved the lab!'. This confirms, from the Web Security

Academy's side as well, that the attack was carried out correctly and the objective was achieved.

Cross-site scripting attacks section conclusion

In this scenario, we found that the application takes data from the user via the search bar and returns it directly into the page's code (HTML). This security gap allows malicious users to inject arbitrary JavaScript code. Although in our example we used a harmless `alert()` command, in real conditions an attacker could use the same vulnerability to steal cookies, leading to the compromise of user accounts.

Bibliography

- PortSwigger. (n.d.). Burp Suite Community Edition. <https://portswigger.net/burp/communitydownload>
- PortSwigger Web Security Academy. (n.d.). Lab: SQL injection vulnerability in WHERE clause allowing retrieval of hidden data. <https://portswigger.net/web-security/sql-injection/lab-retrieve-hidden-data>
- PortSwigger Web Security Academy. (n.d.). Lab: Reflected XSS into HTML context with nothing encoded. <https://portswigger.net/web-security/cross-site-scripting/reflected/lab-html-context-nothing-encoded>
- PortSwigger. (n.d.). SQL injection. <https://portswigger.net/web-security/sql-injection>

- PortSwigger. (n.d.). Cross-site scripting (XSS).
<https://portswigger.net/web-security/cross-site-scripting>
- Seven Seas Security. (2022, May 5). Cross-Site Scripting Lab
Breakdown: Reflected XSS into HTML context with nothing encoded.
<https://www.youtube.com/watch?v=HfBiMStbWqY>
- David Bombal Clips. (2024, June 11). Mastering Burp Suite: the
ultimate web application hacking tool.
<https://www.youtube.com/watch?v=r46b9L6UQDo>
- Netsec Explained. (2023, September 20). Master Burp Suite like a pro
in just 1 hour. https://www.youtube.com/watch?v=QiNLNDSL_uJY